Seguridad de Sistemas Software Prácticas



GRADO EN INGENIERÍA INFORMÁTICA
TECNOLOGÍA ESPECÍFICA. INGENIERÍA DEL SOFTWARE

PROFESOR: LUIS ENRIQUE SÁNCHEZ, DAVID GARCÍA ROSADO

TEMA P1. Requisitos

Seguridad de Sistemas Software

- **Descripción general:**
- Se desea crear un sistema web de **comercio electrónico** que permita a los usuarios registrados buscar, consultar, comprar y valorar productos en un entorno seguro, escalable y accesible desde cualquier navegador web. El sistema debe permitir las opciones básicas de una plataforma de este tipo: administración de productos, gestión de pedidos, gestión del carrito de compra, configuración de productos favoritos, valoraciones de productos y seguimiento de envíos.

Seguridad de Sistemas Software

- **Requisitos generales de seguridad:**

1. Para acceder a la herramienta, los usuarios deberán identificarse ante el sistema con un usuario y contraseña. Para eso debe utilizarse el patrón RBAC.
2. Debe permitir a los usuarios registrarse en el sistema, gestionar sus listas y valorar los contenidos. También podrá gestionar su perfil y su suscripción.
3. Debe permitir a los usuarios gestionar los contenidos de la plataforma, consultar las estadísticas de uso y consultar las valoraciones de los usuarios.

Seguridad de Sistemas Software

- **Requisitos generales:**

4. La aplicación debe tener un módulo de gestión de usuarios, que sea solamente accesible por el administrador. Dicho módulo debe tener las siguientes funciones: Crear (sólo otros administradores), modificar, consultar, bloquear/desbloquear y eliminar usuarios.
5. Debe establecerse una política de contraseñas segura.
6. Debe contar con un registro de usuario seguro y un sistema de recuperación de contraseñas.
7. Debe contar con mecanismos de control y seguridad de sesión.

Seguridad de Sistemas Software

- **Requisitos generales:**

8. Debe ser accesible desde cualquier navegador web.
9. La aplicación contara con una conexión segura del tipo https.
10. En caso de contener información sensible esta deberá estar protegida de forma adecuada en la BBDD.
11. Debe tener el mínimo de vulnerabilidades e identificarlas adecuadamente. Es por ello que el número de defectos, bugs y vulnerabilidades de severidad bloqueantes y críticos debe ser 0.

Seguridad de Sistemas Software

- **Requisitos generales:**

12. Se debe realizar un Análisis y Gestión de riesgos (AGR) del sistema en cumplimiento con la norma ISO27001:2022.
13. Con el objetivo de asegurar el correcto funcionamiento de la aplicación las pruebas que se realicen deben cubrir al menos el 60% del código (mayor al 60%). Por ello, no se permitirá la existencia de código en la aplicación que no sea utilizado.
14. Para poder verificar que nuestro código cumple con estos criterios de calidad y seguridad nos ayudaremos de Sonar y Sonar Lint.

Seguridad de Sistemas Software

- **Requisitos generales:**

15. Se debe cumplir con los principios de seguridad especificados en el documento CCN- CERT BP/28 y en las guías de buenas prácticas vistas en clase.
16. Como medida de protección adicional de seguridad, se deberá contar con un segundo factor de autenticación (ej: Google Authenticator) y un tercer factor (3FA)
17. Se deberán incorporar medidas de seguridad a nivel de BBDD para evitar robo de información en caso de fallo de las capas superiores.
18. Se debe poder hacer una recuperación segura de contraseña.

Seguridad de Sistemas Software

- **Requisitos generales:**

19. En la capa de seguridad, el equipo deberá **utilizar al menos una herramienta de análisis de código asistido por IA para detectar vulnerabilidades de seguridad en el código fuente** (Ej: GitHub CodeQL con Copilot Autofix, Semgrep, Snyk Code o herramientas equivalentes). La herramienta seleccionada deberá integrarse en el flujo de desarrollo. Su uso deberá documentarse indicando, de las vulnerabilidades principales: fecha de análisis, vulnerabilidades detectadas, severidad, propuesta de corrección, decisión adoptada por el equipo. Las recomendaciones generadas por IA no podrán aceptarse de forma automática. Todo cambio deberá ser revisado, entendido, y subsanado. Se entregará un informe final con los resultados.

Aclaraciones sobre requisitos

Seguridad de Sistemas Software

- RQ - Autenticación multifactor (3FA):
  1. Factor 1 (Algo que sabes): Contraseña robusta
     1. Longitud ≥ 8, blacklist de contraseñas comunes, verificación de brechas (opcional), caducidad razonable (30 días - No reutilizar últimas 5).
     2. Bloque progresivo por intentos fallidos + rate limiting
  2. Factor 2 (Algo que tienes): TOTP o llave FIDO2 (factor 2)
     1. TOTP (Google Authenticator, Authy): secreto cifrado en BBDD, QR de alta, ventana de tolerancia ±1 paso, códigos de respaldo.
     2. FIDO2 (llave física/YubiKey o biometría del dispositivo): registro y autenticación WebAuthn (antiphishing).
  3. Factor 3 (Libre, a elegir) (catcha, email, Código único) (factor 3)
     1. Código de un solo uso por e-mail (10 min de validez y hash).
     2. CAPTCHA lo puedes usar como endurecimiento, pero no cuenta como factor NIST.

Seguridad de Sistemas Software

- RQ - Autenticación multifactor (3FA):
  1. Política por rol:
     1. Admin y Creador: 3FA obligatoria (1 + 2 + 3).
     2. Usuario (VIP/Estándar): 3FA opcional; mínimo: Factor 1. Se incentiva activar Factor 2.

Seguridad de Sistemas Software

- RQ – Defensa contra ataques de diccionario y credential stuffing:
  1. Política de contraseñas seguras (≥ 8 caracteres)
     1. Longitud ≥ 8 (recomendable 12+ en producción).
     2. Bloqueo de contraseñas de listas comunes (p. ej., 12345678, Password1, qwertyui).
     3. Evitar datos personales del usuario (nombre, email, empresa, app).

*Aclaración - Qué es - **Credential stuffing** es un tipo de ataque en el que un bot prueba, de forma automática y masiva, pares de usuario-contraseña robados en otros servicios (p. ej., de filtraciones públicas) para entrar en tu aplicación aprovechando la reutilización de contraseñas.*

Seguridad de Sistemas Software

- RQ – Defensa contra ataques de diccionario y credential stuffing:
  1. Rate limiting adaptativo y backoff exponencial – Bloqueo temporal.
     1. *Aclaración - Qué es: limitar/ver espaciar intentos de login y verificación para frenar ataques automáticos; cuando hay abuso, subes la dureza (adaptativo) y aumentas el tiempo de espera tras cada fallo (backoff).*
     2. Base: máx. 5 intentos de login / 5 min por IP+usuario.
     3. Backoff exponencial por cuenta: tras 5 fallos en 10 min → bloquear 30 s; si sigue fallando: 2 min, luego 4 min, luego 5 min.
     4. Adaptativo por IP: si una IP intenta 20 usuarios distintos en 5 min → bajar su umbral a 2 intentos / 5 min y aplicar un retraso fijo (p. ej., 2 s) a cada intento.
     5. Respuesta: HTTP 429 con Retry-After cuando aplique.

Seguridad de Sistemas Software

- RQ – Defensa contra ataques de diccionario y credential stuffing:
  1. Verificación de contraseñas filtradas contra diccionario
     1. *Aclaración - Qué es: Impedir que un usuario se registre o cambie a una clave que ya ha sido filtrada en brechas públicas.*
     2. Mantén un diccionario local (top 1M–10M contraseñas filtradas) y busca en él.
     3. Usar un servicio tipo “k-anonymity” (p. ej., calcular hash de la contraseña y consultar solo el prefijo del hash para no exponerla).
     4. La verificación se realiza en el alta y en el cambio de contraseña.
  2. Hash con Argon2id/bcrypt + pepper
  3. Alertas de intentos fallidos.

Seguridad de Sistemas Software

- RQ – Defensa contra ataques de diccionario y credential stuffing:
  1. Hash con Argon2id/bcrypt + pepper
     1. *Aclaración - Qué es: Almacenar contraseñas solo como hash fuerte con salt único por usuario y un pepper secreto de la aplicación (clave adicional), para resistir robos de BBDD.*
     2. Usar Argon2id:memoryCost: 64–256 MB, iterations: 2–4, parallelism: 1–2 (ajusta al hardware) o Bcrypt: cost (log rounds) 12–14.
     3. Pepper: Un secreto global (env/keystore) concatenado a la contraseña antes de hashear.
     4. Comparación constante para evitar *timing attacks*.
  2. Alertas de intentos fallidos.

Seguridad de Sistemas Software

- RQ – Gestión y control de sesión:
  1. Cookies HttpOnly, Secure, SameSite
     1. HttpOnly: el JS del navegador no puede leer la cookie ⇒ mitiga robo por *XSS*.
     2. Secure: la cookie solo viaja por HTTPS.
     3. SameSite: limita envío de cookies en peticiones *cross-site* (protege frente a *CSRF*).

*Aclaración - Qué es **XSS (Cross-Site Scripting)** es una vulnerabilidad que permite a un atacante inyectar y ejecutar JavaScript en el navegador de la víctima dentro del contexto de tu sitio. Con XSS pueden robar sesiones/cookies, hacer acciones en tu nombre, mostrar formularios falsos (phishing), registrar teclas, etc.*

*Aclaración - Qué es **CSRF (Cross-Site Request Forgery)*** es un ataque que hace que el navegador de un usuario autenticado en tu app envíe peticiones no deseadas a esa misma app desde otro sitio (otra pestaña/web maliciosa).

Seguridad de Sistemas Software

- RQ – Gestión y control de sesión:
  1. Rotación de sesión tras login y privilegios
     1. *Aclaración - Qué es: regenerar el ID de sesión cuando el usuario inicia sesión o sube privilegios, para evitar **session fixation**.*
     2. Rotar: Después de login correcto.
     3. Rotar: Cuando se cambian credenciales.

*Aclaración – Qué es: **Session fixation** es una vulnerabilidad en la que un atacante hace que la víctima use un ID de sesión que el atacante ya conoce; cuando la víctima inicia sesión con ese ID preestablecido, el atacante puede reutilizarlo para tomar su sesión autenticada.*

Seguridad de Sistemas Software

- RQ – Gestión y control de sesión:
  1. Idle timeout 15-30 min, absolute timeout 8-12h
     1. Idle timeout (inactividad): cierra la sesión si no hay actividad durante X minutos (p. ej., 15–30 min).
     2. Absolute timeout: caduca la sesión, aunque haya actividad al llegar a 8–12 h (fin de jornada).

     1. Ejemplo de política:
        1. Usuarios normales: Idle 20 min, Absolute 8 h.
        2. Admin/Creadores: Idle 15 min, Absolute 6–8 h.

Seguridad de Sistemas Software

- RQ – Endurecimiento de frontend y backend:
  1. HTTPS
     1. HTTPS (“HyperText Transfer Protocol Secure”) es HTTP cifrado con TLS. Proporciona un canal seguro entre navegador y servidor.
     2. ¿Qué aporta? Cifrado: nadie puede leer lo que envías/recibes (credenciales, cookies, datos).
     3. Integridad: impide que terceros modifiquen el tráfico en tránsito.
     4. Autenticación del servidor: un certificado (emitido por una CA) prueba que el dominio es quien dice ser.
     5. Validación de entrada y codificación de salida
  2. Rate limiting en peticiones
  3. Gestión de errores genéricos
  4. RBAC con mínimo privilegio

Seguridad de Sistemas Software

- RQ – Endurecimiento de frontend y backend:
  1. Validación de entrada y codificación de salida
     1. Regla de oro: valida al entrar, codifica al salir (contexto correcto).
     2. Valida al entrar (input validation): todo dato que viene del usuario u otra fuente externa se comprueba antes de usarlo o guardarlo: tipo, longitud, formato, rango, lista blanca, tamaño de fichero, etc. (y en servidor, no solo en el front). También se puede normalizar (trim, Unicode NFC) antes de validar.
     3. Codifica al salir (output encoding): cuando vayas a mostrar esos datos, no los renders como crudo; conviértelos al formato seguro según el contexto donde los insertas, para que el navegador no los interprete como código (evita XSS).

Seguridad de Sistemas Software

- RQ – Endurecimiento de frontend y backend:
  1. Rate limiting en peticiones
     1. Objetivo: frenar abuso (fuerza bruta, scraping, spam OTP) y estabilizar el servicio.
     2. Reglas prácticas (ejemplos):
        1. /login: 5 intentos / 5 min por IP+usuario (429 si excede)
        2. /otp/send: 3 envíos / hora / usuario; 10 / día.
        3. API pública: 60 req / min / token.

Seguridad de Sistemas Software

- RQ – Endurecimiento de frontend y backend:
  1. Gestión de errores genéricos
     1. Principio: al cliente, mensaje genérico y seguro; al log, detalle completo con Correlation-ID.
     2. Buenas prácticas:
        1. Nunca expongas stack traces, nombres de tablas, versiones.
        2. Mapea excepciones conocidas:400 (validación), 401 (auth), 403 (permisos), 404 (no existe), 409 (conflicto), 429 (límite), 5xx (genérico).
        3. Logs estructurados (JSON) con: usuario, IP, endpoint, UA, correlationId, causa.

Seguridad de Sistemas Software

- RQ – Endurecimiento de frontend y backend:
  1. RBAC con mínimo privilegio
     1. Qué es: cada endpoint/acción exige el rol/permisos mínimos necesarios.
     2. Por defecto, denegar.
     3. Modelo sugerido:
        1. Roles: USER, CREATOR, ADMIN.
        2. Permisos finos (authorities): POST_CREATE, POST_PUBLISH, USER_SUSPEND, etc.
        3. Propiedad de recurso: además de rol, verifica que el recurso es del usuario cuando aplique.

Seguridad de Sistemas Software

- RQ – Seguridad especifica MongoDB:
  1. Autenticación SCRAM-SHA-256 y roles mínimos
     1. Qué es: usa el mecanismo de autenticación fuerte SCRAM-SHA-256 y otorga a cada servicio solo los permisos que necesita.
     2. Usuarios separados por tarea:
        1. appuser: readWrite sobre su BD.
        2. migrator: readWrite + dbAdmin SOLO en ventanas de despliegue.
        3. Nada de compartir cuentas entre microservicios.

Seguridad de Sistemas Software

- RQ – Seguridad especifica MongoDB:
  1. Cifrado en tránsito (TLS 1.2+)
     1. Qué es: todo tráfico cliente⇄Mongo cifrado.
  2. Prohibir $where y operadores peligrosos o inyecciones de código.
     1. Por qué: $where ejecuta JavaScript en el servidor y abre la puerta a inyecciones y rendimiento pobre. Evita también construir consultas con datos del usuario que contengan claves que empiecen por $ o .

Seguridad de Sistemas Software

- RQ – Subida y entrega de contenidos multimedia:
  1. Validación de tipo MIME y tamaño.
     1. Qué/Por qué: No fiarse de la extensión ni del Content-Type del cliente. Evita ***polyglots*** *(construido para ser válido en dos o más formatos a la vez)* y cargas gigantes.
     2. Límite de tamaño en servidor y proxy (p. ej., 100 MB).
     3. Sniffing por firma (magic numbers) + comprobación de extensión ⇒ lista blanca (ej.: image/jpeg, image/png, video/mp4, audio/aac).
     4. Rechazar tipos ambiguos (text/html, application/javascript, application/octet-stream salvo casos muy concretos).
  2. Transcodificación en sandbox.
  3. Escaneo antivirus.
  4. Cabeceras seguras en descargas.
  5. URLs firmadas y caducidad controlada.

Seguridad de Sistemas Software

- RQ – Subida y entrega de contenidos multimedia:
  1. Escaneo antivirus (opcional)
     1. Qué/Por qué: algunos formatos permiten cargas con contenido malicioso.
  2. Cabeceras seguras en descargas (opcional)
  3. URLs firmadas y caducidad controlada (opcional)
     1. Qué/Por qué: el objeto no es público; solo accesible mediante enlaces temporales.

Seguridad de Sistemas Software

- RQ – Cumplimiento y privacidad:
  1. Gestión de secretos y claves.
  2. Registros de auditorias securizados.
  3. AGR según ISO/IEC 27001:2022 con MARISMA.
  4. Cumplimiento RGPD
  5. Data masking y anonimización de logs

Documentación

Seguridad de Sistemas Software

- Documentación:
  - Se debe entregar de cada requisito, una explicación detallada de como se ha solucionado y un fragmento del código implementado para solucionar el requisito.
  - Se valorara la presentación y calidad de la documentación.